# Hyperspectral Image Classification of Brain-Inspired Spiking Neural Network Based on Approximate Derivative Algorithm

Yang Liu, *Member, IEEE*, Kejing Cao, Rui Li, Hongxia Zhang, and Liming Zhou

***Abstract*— Recently, deep learning methods have made significant progress in solving hyperspectral image (HSI) classification problems of high-dimensional features, band redundancy, and spectral mixture. However, the deep neural network is too complex, with a long training time and high energy consumption, making it difficult to deploy on edge computing devices. In order to solve the above problems, this article proposes a brain-inspired computing framework based on the spiking leaky integrate-and-fire neuron model for HSIs' classification. Then, we design an approximate derivative algorithm to solve the nondifferentiable spike activity of the spiking neuron. The framework uses direct coding to generate spatiotemporal spikes for input HSI and achieves efficient extraction of spatial–spectral features through spiking standard convolution and spiking depthwise separable convolution. Extensive experiments are performed on four benchmark hyperspectral datasets and two public unmanned aerial vehicle-borne hyperspectral datasets. Experiments show that the proposed model has the advantages of high classification accuracy and fewer spiking time steps. The proposed model can save about ten times computational energy consumption compared with the CNN of the same architecture. This research has great significance for overcoming the technical bottleneck of HSI classification based on brain-inspired computing, solving the critical problems of mobile computing in unmanned autonomous systems, and realizing the engineering application of unmanned aerial vehicles and software-defined satellites. The source code will be made available at https://github.com/Katherine-Cao/HSI_SNN.**

***Index Terms*— Approximate derivative algorithm, brain-inspired computing, hyperspectral image (HSI) classification, neuromorphic computing, spiking neural network (SNN).**

## I. Introduction

AS AN essential part of Earth-observing technology, remote sensing technology has been widely used in forest management [1], soil salinity estimation [2], mineral exploration [3], anomaly detection [4], military reconnaissance [5], urban planning [6], road extraction [7], and land-user change detection [8]. In recent years, with the development of remote sensing technology and hyperspectral imaging technology, hyperspectral images (HSIs) contain richer spectral information and higher spatial resolution. HSIs provide the possibility for accurate classification and recognition of complex ground targets.

HSI classification is a kind of pixel-based classification, which mainly distinguishes different categories of each pixel in the image according to the information carried by different bands. Most early HSI classification techniques used a combination of classifiers and dimension reduction. These methods only used the spectral features of ground objects and ignored their spatial features. Common classifiers mainly include support vector machine (SVM) [9], 3-D wavelet transform [10], and random forest [11]. Meanwhile, in order to solve the problem of dimensionality disaster caused by the high dimensionality and redundancy of data, the researchers mostly reduce the dimensionality of the data through feature selection and feature extraction. Common feature extraction methods include principal component analysis (PCA) [12], independent component analysis, and linear discriminant analysis.

With the promotion of deep learning in image processing, speech recognition, and natural language processing, some researchers have begun to apply deep learning theories and techniques to realize HSIs' classification. Chen *et al.* [13] constructed stacked autoencoders (SAEs), reduced the dimensionality of the original HSIs through PCA operation, and input the extracted features into SAE. Compared with the traditional methods, this method has made a significant breakthrough, but all layers of the SAE network are fully connected so that it will generate more training parameters. Hu *et al.* [14] proposed a convolutional neural network (CNN) to classify HSIs for the first time, but this method only considered spectral features and ignored the spatial relationship among pixels. Makantasis *et al.* [15] used randomized PCA to reduce the number of spectral channels of the input HSIs and constructed a 2-D CNN and MLP combined model to extract the spatial–spectral information of HSIs. Roy *et al.* [16] extracted the features of the input HSIs through PCA and proposed a network architecture consisting of a spectral–spatial 3-D CNN and a spatial 2-D CNN to achieve HSIs' classification. Then,

Manuscript received 4 August 2022; revised 9 September 2022; accepted 12 September 2022. Date of publication 15 September 2022; date of current version 29 September 2022. This work was supported in part by the National Natural Science Foundation of China under Grant 62176087, in part by the Shenzhen Special Foundation of Central Government to Guide Local Science & Technology Development under Grant 2021Szvup032, in part by the Postgraduate Education Reform and Quality Province Improvement Project of Henan Province under Grant YJS2022JC33, and in part by the Research and Practice Project of Teaching Reform in Henan University under Grant HDXJJG2020-109. *(Corresponding authors: Liming Zhou; Yang Liu.)*

The authors are with the Henan Province Engineering Research Center of Spatial Information Processing, the Henan Key Laboratory of Big Data Analysis and Processing, the School of Computer and Information Engineering, and the Shenzhen Research Institute, Henan University, Kaifeng 475004, China (e-mail: ly.sci.art@gmail.com; katherine_cao@henu.edu.cn; 958105892@qq.com; 1748423724@qq.com; lmzhou@henu.edu.cn).

Digital Object Identifier 10.1109/TGRS.2022.3207098

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

3-D CNNs are widely used in HSIs' classification tasks and have achieved good classification results. Zhong *et al.* [17] proposed a 3-D CNN, and the model achieved high-precision classification through the residual network structure. Based on the principle of the pixel-level HSIs' classification and the definition of spatial information, Liu *et al.* [18] proposed a central attention network to effectively extract spectral–spatial features. As an extension of CNN, the graph convolutional network (GCN) has also been used in HSIs' classification in recent years. Hong *et al.* [19] developed a new version of supervised GCN and integrated the GCN version with CNN to achieve a high classification accuracy. In addition, transformer [20] has also been used in HSIs' classification. Sun *et al.* [21] combined a transformer with CNN to capture spectral–spatial features and high-level semantic features. Hong *et al.* [22] proposed a new transformer-based backbone network to focus on extracting spectral information. Most current hyperspectral classification models are patch-based learning frameworks, which incur additional computational overhead. To address this issue, Zheng *et al.* [23] proposed a fast patch-free global learning (FPGA) framework to achieve the end-to-end classification of HSIs. Zhu *et al.* [24] designed a spectral–spatial-dependent global learning framework to extract deep spectral–spatial features.

In order to fully extract the spatial–spectral features of HSIs, the depth of the classification model is gradually deepened, and the design of the classification model is gradually complicated. This leads to excessive network parameters, long training, and inference times. At the same time, an HSI cube contains many spectral brands, all of which aggravate the computational energy required for classification. Although many researchers have proposed lightweight HSI classification models [25], [26], these models are all deep neural network models, and the calculation method of the network is continuous calculation [27], which still consumes a lot of energy in image classification. As computational energy costs continue to increase and the number of energy-constrained applications (autonomous systems, robotics, remote sensing, and satellites) grows [28], researchers need to explore low-energy classification models. In addition, edge computing devices, such as satellites and unmanned aerial vehicles, provide rich hyperspectral data. With the continuous advancement of remote sensing technology and sensor technology, models can be deployed on HSI sensors in such devices in the future to solve real-time image processing. In order to solve the above problems, this article strives to explore a high-performance, low-energy network model to achieve HSIs' classification.

Inspired by biological neural networks, which have significant advantages in energy consumption, parallel processing, and spatiotemporal computing, researchers realize neuromorphic computing or brain-inspired computing by constructing a brain-inspired spiking neural network (SNN) [29], [30], [31]. Compared with the artificial neural network (such as CNN), the design principle of SNN is closer to the brain-inspired mechanism. SNN uses binary spike trains to realize the interaction of information and considers the spatiotemporal information in the training and inference process. The spiking neurons that construct the SNN will be in an active state when receiving a spike and will be in a resting state at other times so that SNN can save enormous computational energy compared with the artificial neural network of continuous computing.

In recent years, many methods have emerged for training SNNs, among which supervised training methods are generally divided into two categories. One is to train an artificial neural network, save its network weights, and then add spiking neurons to the artificial neural network to convert it into an SNN with the same architecture. However, the classification performance of this method cannot surpass the artificial neural network, and this method does not explore the nature of SNNs [32], [33], [34]. Another approach is to directly train an SNN [35], [36], [37], [38], [39]. The optimization of parameters in the direct training process of the artificial neural network is mainly based on the error backpropagation algorithm (BP algorithm). However, the difference between SNNs and other artificial neural networks is that the inside of the spiking neurons is discontinuous spike trains. The discrete spiking activity makes it impossible to directly use the BP algorithm to optimize the SNN network parameters. Therefore, researchers mainly optimize the network parameters through approximate derivative algorithms to achieve the training and inference of SNNs. Wu *et al.* [36] constructed a spatiotemporal BP (STBP) training framework based on the LIF neuron model and proposed four approximate derivative functions to solve the nondifferentiable problem. Neftci *et al.* [37] introduced surrogate gradient (SG) methods that addressed the problem of nondifferentiable functions. Lee *et al.* [38] processed the derivative of the LIF neuron into two parts. First, the approximate derivative of the IF neuron that does not contain the leakage characteristic is used, and then, the corresponding leakage compensation is added to the LIF neuron to realize the direct training of the SNN. Wu *et al.* [31] used the approximate derivative function proposed in the literature [36] to design a brain-inspired SNN model and put it on the neuromorphic platform Tianjic chip. Based on thoroughly studying the work of relevant researchers, this article proposes an approximate derivative algorithm to realize the direct training of SNNs.

Currently, the classification tasks of SNNs are mainly focused on benchmark datasets, such as MINST and CIFAR-10. In order to promote the combination of mobile computing or edge computing and HSI classification tasks, we propose a low-energy brain-inspired SNN. The contributions of this article are summarized as follows.

1) The SNN framework is based on the spiking leaky integrate-and-fire (LIF) neuron model. Aiming at the nondifferentiable issue of the spiking activity in the LIF neuron, we proposed an approximate derivative algorithm based on the principle of LIF neuron dynamics. The SNN model optimizes network parameters through the approximate derivative algorithm to achieve high-precision HSIs' classification.
2) The model framework is mainly composed of the spiking encoder layer, the spiking standard convolution layer, and the spiking depthwise separable convolution layer. The spiking encoder layer receives the input pixel values of HSIs and converts them into spike trains. The spiking standard convolution layer fully extracts the

spatial–spectral features of HSIs. The spiking depthwise separable convolution layer further reduces the network parameters and computational cost while extracting the spatial–spectral features of HSIs.

3) The optimal classification parameters of the SNN model are determined by four well-known hyperspectral datasets (Indian Pines (IP), Pavia University, Salinas, and Houston2013) and two public hyperspectral datasets of WHU-Hi [40] captured by unmanned aerial vehicle-borne hyperspectral systems. In addition, we compare the proposed model with the mainstream HSIs' classification models and verify the effectiveness of the approximate derivative algorithm and model architecture. By calculating the number of spiking neurons activated in each layer and the number of total operations in the model, it can be seen that the proposed model can save about ten times computational energy consumption compared with the CNN of the same architecture.

The rest of this article is structured as follows. Section II presents the related approaches to building a brain-inspired computing framework and the model architecture details. Section III determines the optimal classification parameters of the proposed model on six hyperspectral datasets through experiments. The comprehensive and objective experiments and analysis are presented in Section IV. Finally, Section V summarizes this article.

## II. Approach

In this section, we introduce the related approaches of constructing a brain-inspired SNN for HSI classification one by one and the details of the final model architecture.

### A. Leaky Integrate-and-Fire Neuron Model

The human brain is a complex network with billions of biological neurons and trillions of synapses. The spike-based processing mechanism makes the biological neurons only consume energy when working, so the computational energy consumption is extremely low. In the neural system, the neuron receives and stores the spike from the preneuron and changes or remembers the synapses' strength between two neurons. As time steps accumulate, the spikes continue to increase. A spike will be released when the neuron's membrane potential exceeds the threshold. Then, the spike is delivered to the postneurons through the synapse. During this process, there is a certain amount of leakage of the membrane potential over time. Inspired by biological neurons, many researchers have established neuron models, including the Hodgkin–Huxley model [41], the LIF model [42], and the Izhikevich model [43]. Among them, the LIF model simplifies the neuron's working process and retains its three essential characteristics (leaking, integrating, and firing). The formula of the membrane potential $V$ of the LIF model is as follows:

$$\tau_m \frac{dV}{dt} = -V + I(t) \tag{1}$$

where $\tau_m$ is the constant of membrane potential decay with time and $I$ is the input current. In order to enable the LIF neuron to accumulate more spike signals over short time steps, this article uses the improved LIF neuron model based on previous research work [39]. The formula of the improved LIF neuron membrane potential $v(t)$ is shown in the following, and the forward propagation process of the LIF neuron model is shown in Fig. 1:

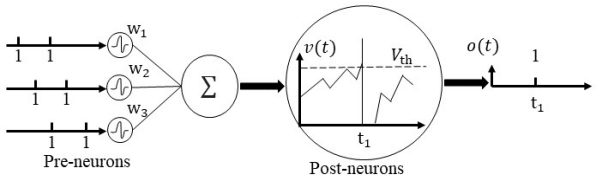



Fig. 1. Illustration of the LIF neuron dynamics.

$$v_i^t = \left(1 - \tau_m'\right) v_i^{t-1} + \tau_m' \sum_j w_{ij} o_j^t, \quad \tau_m' \in (0 \sim 1) \tag{2}$$

$$o_j^t = \begin{cases} 0, & v_j^t < V_{\text{th}} \\ 1, & v_j^t \geq V_{\text{th}} \end{cases} \tag{3}$$

where $\tau_m'$ is the constant of membrane potential decay with time and $w_{ij}$ represents the synaptic weight between neuron $i$ and neuron $j$. Formula (2) represents the membrane potential of the $i_{\text{th}}$ neuron at time step $t$. The membrane potential $v_i^t$ during the forward propagation is composed of the membrane potential $v_i^{t-1}$ that did not exceed the threshold at the previous time step $t-1$ and the output spikes generated by preneurons at the current time step $t$. In addition, these two parts realize the leakage of membrane potential according to a certain proportion. In formula (3), if the membrane potential $v_j^t$ of the $j_{\text{th}}$ neuron exceeds the threshold $V_{\text{th}}$ at time step $t$, an output spike $o_j^t$ will be released. Here, $o_j^t = 1$ means that the $j_{\text{th}}$ neuron at time step $t$ has spike activity, and $o_j^t = 0$ means that the $j_{\text{th}}$ neuron at time step $t$ has no events. When a neuron releases a spike signal, its membrane potential will return to a resting state until the next spike signal arrives.

The neurons in the last layer of the network will receive all of the forward output spikes and divide the membrane potential ($V$) of the last layer by the total time steps (time steps) to get the final output (output)

$$\text{output} = \frac{V(t)}{\text{time steps}}. \tag{4}$$

### B. Approximate Derivative Algorithm

This article defines the loss function $L$ to reduce the difference between the predicted output and the label, and updates the weight $w$ of each layer according to the chain derivative rule. The derivative of the weight parameter $w$ can be expressed as

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial O} \frac{\partial O}{\partial V} \frac{\partial V}{\partial w} \tag{5}$$

where $L$ represents the loss function, $w$ represents the network weight parameter, $V$ represents the neuron's membrane potential, and $O$ represents the output spikes after the neuron is activated. Since the release of the output spikes is discrete,

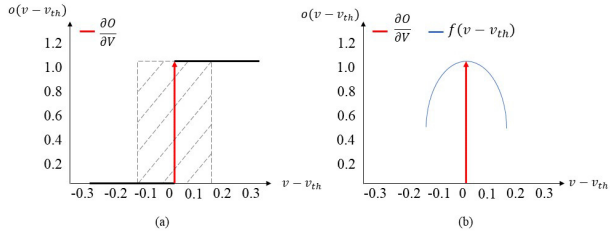



Fig. 2. Illustration of the nondifferentiable spike activity. (a) Output spikes' function $O(v - v_{\text{th}})$ and the original derivative $(\partial O/\partial V)$ of spike activity at firing time. (b) Curve function $f(v - v_{\text{th}})$ to approximate the output spikes' function $O(v - v_{\text{th}})$ of spike activity at firing time.

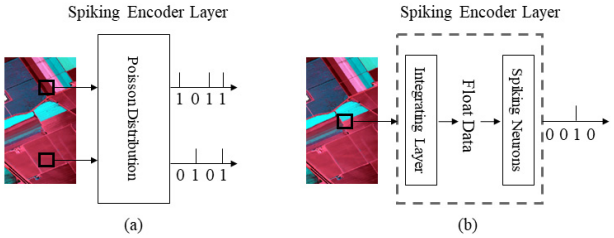



Fig. 3. Illustration of (a) rate coding and (b) direct coding over four time steps.

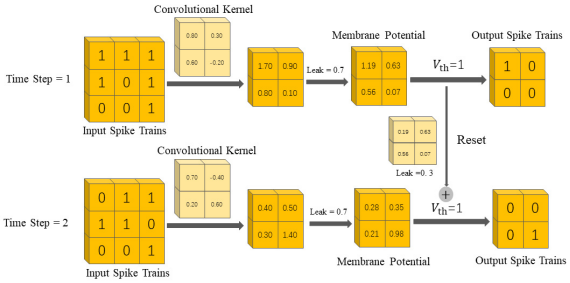



Fig. 4. Illustration of the simplified spiking convolution operation (assuming 2-D input and 2-D weight kernel) over two time steps.

it is not possible to directly optimize the network parameters through the BP algorithm in the training of the SNN, so $(\partial O/\partial V)$ in formula (5) cannot be directly derived. Based on the dynamics of spiking LIF neurons, this article proposes a curve function $f(v - v_{\text{th}})$ to approximate the output spikes' function $O(v - v_{\text{th}})$ to optimize the network parameters. The specific details are shown in Fig. 2. $O(v - v_{\text{th}})$ means that, when $v - v_{\text{th}}$ is greater than 0, a spike will be released, which will cause the gradient to disappear or the gradient to explode during the calculation of the derivative. The formula $f(v-v_{\text{th}})$ is given as follows:

$$f(v - v_{th}) = \left|1 - (v - v_{th})^2\right| < \delta, \quad \delta \in (0 \sim 1] \tag{6}$$

where $\delta$ is a custom parameter and the function $f(v - v_{\text{th}})$ has the same function value as the function $O(v - v_{\text{th}})$ when $v - v_{\text{th}}$ approaches zero at the spike activity

$$\lim_{(v-v_{th})\to 0^+} f(v - v_{th}) \approx \lim_{(v-v_{th})\to 0^+} O(v - v_{th}). \tag{7}$$

We replace the derivative $(\partial O/\partial V)$ by derivative $(\partial f/\partial V)$. That is, $(\partial O/\partial V)$ can be approximate as

$$\frac{\partial O}{\partial V} \approx \frac{\partial f}{\partial V}. \tag{8}$$

The optimization of network weight $w$ in formula (5) is approximate

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial O}\frac{\partial O}{\partial V}\frac{\partial V}{\partial w} \approx \frac{\partial L}{\partial O}\frac{\partial f}{\partial V}\frac{\partial V}{\partial w}. \tag{9}$$

The model architecture of this article is based on the approximate derivative algorithm to optimize network parameters. To verify the effectiveness of this algorithm, we conduct extensive and objective experiments in Sections III and IV.

### C. Spiking Encoder Schemes

In order to realize HSI classification, the SNN first converts the input images into binary spike trains through the spiking encoder layer and iterates multiple time steps to complete the superposition of the spike trains. The current mainstream spiking encoder schemes are divided into rate coding and direct coding. Rate coding is coded according to the firing frequency of the spike trains, where the firing frequency is proportional to the intensity of the input pixel values. Most of the current image classification work of the SNN adopts this coding scheme [44], [45], [46]. Fig. 3(a) shows the details of rate encoding. Direct coding [47] directly receives the input image through the integrating layer, generates a floating-point output, and then converts the floating-point number into a binary spike through a spiking neuron. Fig. 3(b) shows the details of the direct coding. A recent study [48] systematically analyzed these two coding schemes in terms of accuracy, robustness, and energy efficiency. Through experiments, it was found that direct coding can achieve higher classification accuracy over shorter time steps. Therefore, we adopt direct coding as the coding scheme of the spiking encoder layer.

### D. Spiking Convolution Operation

The input HSIs first pass through the spiking encoder layer to generate spatiotemporal spikes. Then, the spike trains go through the spiking standard convolution layer or the spiking depthwise separable convolution layer for convolution operation. The specific details of the spiking convolution operation are shown in Fig. 4. When the time step is one, the spike trains from the preneurons are convolved with the convolution kernel, and generated values are stored in the postneurons. When the values exceed the membrane potential threshold $V_{\text{th}}$, output spike trains will be generated. The spike trains that do not exceed the threshold will continue to be stored in the postneurons, adding to the spikes arriving at the next time step. The framework in this article is based on LIF neurons, so the input spikes and the spike trains stored in the neuron during the convolution operation will leak in a certain proportion.

### E. Spiking Standard Convolution and Spiking Depthwise Separable Convolution

Fig. 5 shows the rough details of the spiking standard convolution. In the spiking standard convolution, the number

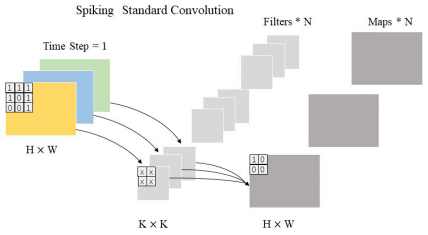



Fig. 5. Illustration of the spiking standard convolution operation (assuming that the input channel is 3).

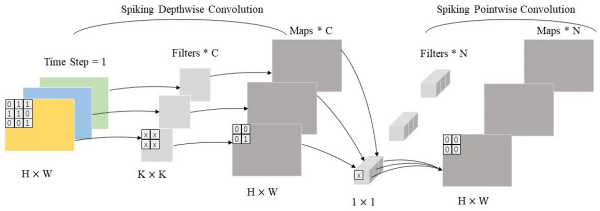



Fig. 6. Illustration of the spiking depthwise separable convolution operation (assuming that the input channel is 3).

of channels of the input HSIs is the same as the number of channels of a single convolution kernel, and each channel needs to be convolved with the corresponding channels of the $N$ convolution kernels. Fig. 6 shows the rough details of the spiking depthwise separable convolution. The spiking depthwise separable convolution consists of depthwise convolution and pointwise convolution. Each channel of the input HSIs in the depthwise convolution is only convolved with a single channel of the convolution kernel and then the generated feature maps. Each feature map generated in the depthwise convolution process performs convolution operations with the corresponding channels of pointwise convolution kernels to generate new feature maps, thereby realizing information fusion of different channels. The convolution principle of pointwise convolution is similar to standard convolution, but the convolution kernel size is $1 \times 1$.

In Figs. 5 and 6, we only consider the convolution operation process over one time step. For the details of leakage, integrating, and firing inside the neurons during the convolution process, please refer to Fig. 4. Compared with spiking standard convolution, spiking depthwise separable convolution can significantly reduce the computational complexity and the number of parameters under certain conditions. In the standard convolution, the number of operations for a convolution operation is $H \times W \times C \times N \times K \times K$, where $H$, $W$, and $C$ represent the length, width, and height of the input HSI, respectively, $K$ represents the convolution kernel size, and $N$ represents the number of the convolution kernel. The number of operations of a convolution operation in depthwise separable convolution is $H \times W \times C \times K \times K + H \times W \times C \times N \times 1 \times 1$. Therefore, the number of operations of depthwise separable convolution can be eight to nine times less than that of standard convolution in theory.

TABLE I

IMPLEMENTATION DETAILS OF THE PROPOSED MODEL (ASSUMING THAT THE SPATIAL SIZE IS 13)

| Layer Name | | Input |
|---|---|---|
| Input Layer | | Hyperspectral image pixel block after PCA processing |
| Spiking Encoder Layer | SConv-LIF Neuron | 13×13,30 (K=3 S=1) |
| BLOCK_1 | SConv-LIF Neuron | 13×13,32 (K=3 S=1) |
| | SConv-LIF Neuron | 13×13,64 (K=3 S=1) |
| BLOCK_2 | DConv-LIF Neuron | 13×13,32 (K=3 S=1) |
| | PConv-LIF Neuron | 13×13,32 (K=1 S=1) |
| Channel Shuffle | | 13×13,96 |
| SConv-LIF Neuron | | 13×13,96 (K=3 S=1) |
| Average Pooling | | 13×13,128 (K=2) |
| BLOCK_3 | SConv-LIF Neuron | 6×6,64 (K=3 S=1) |
| | SConv-LIF Neuron | 6×6,128 (K=3 S=1) |
| BLOCK_4 | DConv-LIF Neuron | 6×6,64 (K=3 S=1) |
| | PConv-LIF Neuron | 6×6,64 (K=1 S=1) |
| Channel Shuffle | | 6×6,192 |
| SConv-LIF Neuron | | 6×6,192 (K=3 S=1) |
| Average Pooling | | 6×6,256 (K=2) |
| Spiking Dncoder Layer | Full Connected | 3×3,256 |
| Output Layer | | Classification Maps |

### *F. Framework of the Proposed Model*

The human brain performs outstanding tasks (e.g., recognizing multiple objects simultaneously, reasoning, controlling, and moving) by receiving and firing spikes. The working state of its internal neurons can be roughly summarized as follows: the current neurons receive the spike signals from the preneurons, and the spike signals are accumulated inside the neuron. When the membrane potential of the neurons exceeds the threshold, the neurons will release spikes, and the spikes are delivered to the postneurons through the synapse. In addition, the neuron's membrane potential also leaks over time.

Based on this brain-inspired principle, this article constructs an SNN framework to classify HSIs based on the direct coding scheme and LIF neuron model. The model architecture is shown in Fig. 7, and the details of the model parameters are shown in Table I. Where SConv represents spiking standard convolution, DConv represents spiking depthwise convolution, PConv represents spiking pointwise convolution, $K$ represents the kernel size, and $S$ represents the stride.

The framework is mainly composed of the input layer, the spiking encoder layer, the spiking standard convolution and depthwise separable convolution layer, the average pooling layer, and the spiking decoder layer. First, a part of useful or valuable features are extracted from the original HSIs through PCA operation, and the features take any pixel point as the center and intercept a pixel block of appropriate size. The pixel block then generates spike trains of 0 or 1 by the spiking encoder layer over $T$ time steps. The output spike trains are divided into two parts and sent to the two submodules of the network, respectively. The spike trains of the left branch pass through two spiking standard convolution layers with the convolution kernel size of $3 \times 3$ successively. The features of HSIs can be fully extracted through a small convolution kernel size. The specific details are shown in Figs. 4 and 5; the spike trains of the right branch pass through the spiking depthwise convolution layer with the convolution kernel size of $3 \times 3$ and the spiking pointwise convolution

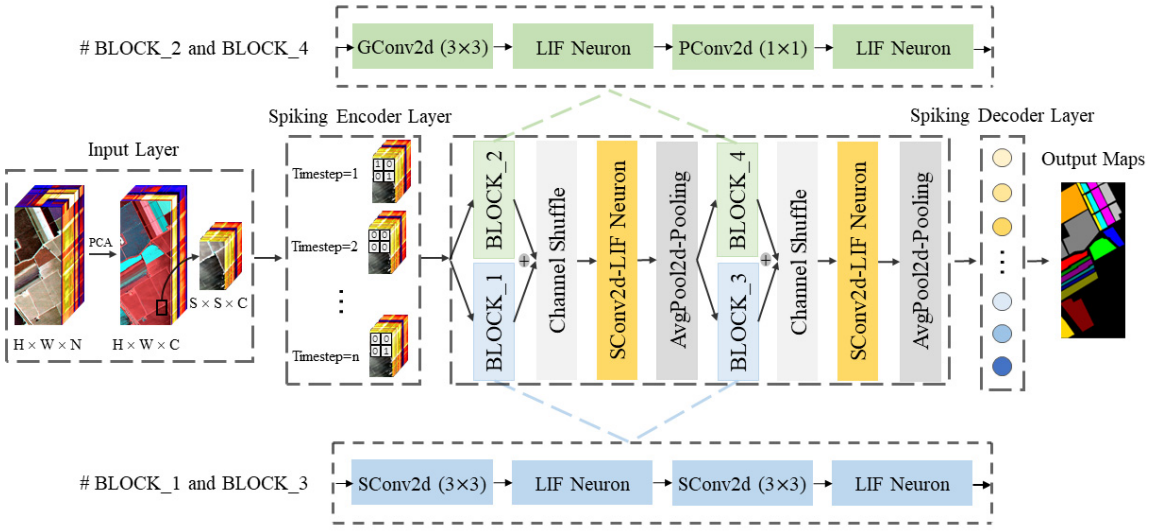



Fig. 7. Framework of the proposed model (BLOCK_1 and BLOCK_3 consist of spiking standard convolution (SConv) layers and LIF neurons; BLOCK_2 and BLOCK_4 consist of spiking depthwise separable convolution (DConv and PConv) layers and LIF neurons).

layer with the convolution kernel size of 1 × 1 successively. This branch can greatly reduce the number of operations for the proposed model to extract HSIs' features. The specific details are shown in Figs. 4 and 6. A discussion of standard convolution and depthwise separable convolution operations is given in Section II-E. Next, the model concatenates the two-part output spike trains representing the HSIs' features. In order to fully fuse the extracted image features, the model shuffles the spike trains through channel shuffle. Then, the spike trains pass through the spiking standard convolution layer with the convolution kernel size of 3 × 3 to fully extract HSIs' features and the average pooling layer to reduce the size of the input HSIs to speed up the running speed of the model. Next, the spike trains are divided into two parts again and sent to two branches, respectively. The above steps are repeated. Finally, the input HSIs pass through the spiking decoder layer to obtain the final classification results.

Since the spike trains representing HSIs' features inside the LIF neurons that make up the framework are discrete, the optimization of network model parameters cannot directly use the BP algorithm. Therefore, the optimization of model parameters in this article adopts the approximate derivative algorithm proposed in Section II-B. In order to verify the effectiveness of the algorithm and classification model, we conducted a large number of experiments on six public hyperspectral datasets. The experimental part will be described in Sections III and IV. In addition, to ensure the fairness of the experiments, each group of experiments is repeated ten times.

## III. Experimental Setup and Parameter Discussion

In this section, we introduce the details of hyperspectral datasets and experimental parameter settings, and obtain the proposed model's optimal classification parameters (spatial size and time step) through experiments on six hyperspectral datasets.

TABLE II
Detailed Information of Six Hyperspectral Datasets

| | IP | PU | SA | HU | WHHC | WHLK |
|---|---|---|---|---|---|---|
| Sensor | AVIRIS | ROSIS | AVIRIS | CASI 1500 | Nano-Hyperspec | Nano-Hyperspec |
| Image Size | 145×145 | 610×340 | 512×217 | 349×1905 | 1217×303 | 550×400 |
| Label Ratio | 48.75% | 17.43% | 48.72% | 2.26% | 69.84% | 92.97% |
| Time | 1992 | 2003 | 1998 | 2012 | 2016 | 2018 |
| Bands | 224 | 102 | 224 | 144 | 274 | 270 |
| Num. of Classes | 16 | 9 | 16 | 15 | 16 | 9 |

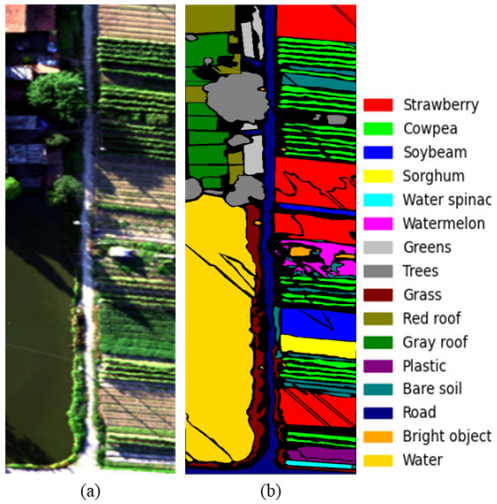



Fig. 8. WHHC dataset. (a) Three-band false color composite image. (b) Ground-truth image.

### A. Datasets

In this section, four benchmark hyperspectral datasets, IP, University of Pavia (PU), Salinas (SA), and Houston2013

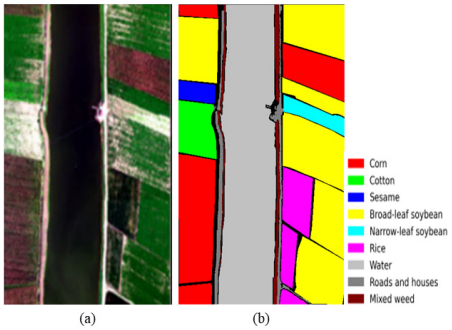



Fig. 9. WHLK dataset. (a) Three-band false color composite image. (b) Ground-truth image.

(HU), and two public hyperspectral datasets, WHU-Hi-HanChuan (WHHC) and WHU-Hi-LongKou (WHLK), are captured by unmanned aerial hyperspectral systems to verify the effectiveness of the framework. The first three datasets can be obtained through the website (http://www.ehu.eus/ccwintco/index.php/Hyperspectral_Remote_Sensing_Scenes). The HU dataset was used for the 2013 IEEE GRSS data fusion contest, available through (https://hyperspectral.ee.uh.edu/?page_id=459), and the latter two datasets can be obtained through the website (http://rsidea.whu.edu.cn/resource_WHUHi_sharing.htm). The six datasets were captured in Northwestern Indiana, Pavia, Italy, California, USA, the campus of the University of Houston in the USA, Hanchuan, China, and Longkou Town, China. The details of the six hyperspectral datasets are shown in Table II. The three-band false color composite image of WHHC and WHLK datasets and the corresponding ground-truth image are shown in Figs. 8 and 9.

### *B. Experimental Settings*

*1) Division of the Datasets:* 200 samples are randomly selected as training sets in six hyperspectral datasets. In the IP dataset, there is a phenomenon that a single sample is less than 200, so 80% of samples are randomly selected as training sets in the experiment, and the rest are used as test sets. The specific division details of the six datasets are shown in Tables III–VIII.

*2) Experimental Conditions and Parameter Settings:* The proposed model uses the deep learning framework of Python version 3.6 and PyTorch version 1.3. Referring to the work of [16], IP, SA, HU, and WHHC datasets retain 30 dimensions through PCA operation, and PU and WHLK datasets retain 20 dimensions through PCA operation. The batch size in the experiment is set to 64, the learning rate is uniformly set to 0.0085, and the weight decay is set to 0.0001. The experiment uses the stochastic gradient descent (SGD) optimizer and the cross-entropy loss function to optimize the network parameters. The experiments repeat ten times, and each time is performed for 100 epochs. During the experiments, the parameters $\tau_m$ and $\delta$ are both set to 0.7.

*3) Analysis of the Experiments:* In this article, the overall accuracy (OA), the average accuracy (AA), and the kappa coefficient ($K$) are used as the evaluation indicators of the experimental results. In order to ensure the accuracy of the experiments, all of the experiments are performed ten times in each group, and the average and standard deviation of the results of the ten-time experiments are taken as the final experimental results. In order to prevent the order of training

TABLE III
SAMPLES' INFORMATION FOR THE IP DATASET

| Class NO. | Class Name | Total | Train | Test |
|---|---|---|---|---|
| 1 | Alfalfa | 46 | 37 | 9 |
| 2 | Corn-notill | 1428 | 200 | 1228 |
| 3 | Corn-mintill | 830 | 200 | 630 |
| 4 | Corn | 237 | 200 | 37 |
| 5 | Grass-pasture | 483 | 200 | 283 |
| 6 | Grass-trees | 730 | 200 | 530 |
| 7 | Grass-pasture-mowed | 28 | 23 | 5 |
| 8 | Hay-windowed | 478 | 200 | 278 |
| 9 | Oats | 20 | 16 | 4 |
| 10 | Soybean-notill | 972 | 200 | 772 |
| 11 | Soybean-mintill | 2455 | 200 | 2255 |
| 12 | Soybean-clean | 593 | 200 | 393 |
| 13 | Wheat | 205 | 200 | 5 |
| 14 | Woods | 1265 | 200 | 1065 |
| 15 | Bulidings-Grass-Trees | 386 | 200 | 186 |
| 16 | Stone-Steel-Towers | 93 | 75 | 18 |
| | Total | 10249 | 2551 | 7698 |

TABLE IV
SAMPLES' INFORMATION OF TRAINING AND TESTING FOR EACH CATEGORY FOR THE PU DATASET

| Class NO. | Class Name | Total | Train | Test |
|---|---|---|---|---|
| 1 | Asphalt | 6631 | 200 | 6431 |
| 2 | Meadows | 18649 | 200 | 18449 |
| 3 | Gravel | 2099 | 200 | 1899 |
| 4 | Trees | 3064 | 200 | 2864 |
| 5 | Sheets | 1345 | 200 | 1145 |
| 6 | Bare soils | 5029 | 200 | 4829 |
| 7 | Bitumen | 1330 | 200 | 1130 |
| 8 | Bricks | 3682 | 200 | 3482 |
| 9 | Shadows | 947 | 200 | 747 |
| | Total | 42776 | 1800 | 40976 |

TABLE V
SAMPLES' INFORMATION OF TRAINING AND TESTING FOR EACH CATEGORY FOR THE SA DATASET

| Class NO. | Class Name | Total | Train | Test |
|---|---|---|---|---|
| 1 | Brocoli-green-weeds-1 | 2009 | 200 | 1809 |
| 2 | Brocoli-green-weeds-2 | 3726 | 200 | 3526 |
| 3 | Fallow | 1976 | 200 | 1776 |
| 4 | Fallow-rough-plow | 1394 | 200 | 1194 |
| 5 | Fallow-smooth | 2678 | 200 | 2478 |
| 6 | Stubble | 3959 | 200 | 3759 |
| 7 | Celery | 3579 | 200 | 3379 |
| 8 | Grapes-untrained | 11271 | 200 | 11071 |
| 9 | Soil-vinyard-develop | 6203 | 200 | 6003 |
| 10 | Gorn-seneced-gree-weed | 3278 | 200 | 3078 |
| 11 | Lettuce-romaine-4wk | 1068 | 200 | 868 |
| 12 | Lettuce-romaine-5wk | 1927 | 200 | 1727 |
| 13 | Lettuce-romaine-6wk | 916 | 200 | 716 |
| 14 | Lettuce-romaine-7wk | 1070 | 200 | 870 |
| 15 | Vinyard-untrained | 7268 | 200 | 7068 |
| 16 | Vinyard-vertical-trelis | 1807 | 200 | 1607 |
| | Total | 54129 | 3200 | 50929 |

TABLE VI

SAMPLES' INFORMATION OF TRAINING AND TESTING FOR EACH CATEGORY FOR THE WHHC DATASET

| Class NO. | Class Name | Total | Train | Test |
|---|---|---|---|---|
| 1 | Strawberry | 44735 | 200 | 44535 |
| 2 | Cowpea | 22753 | 200 | 22553 |
| 3 | Soybean | 10287 | 200 | 10087 |
| 4 | Sorghum | 5353 | 200 | 5153 |
| 5 | Water spinach | 1200 | 200 | 1000 |
| 6 | Watermelon | 4533 | 200 | 4333 |
| 7 | Greens | 5903 | 200 | 5703 |
| 8 | Trees | 17978 | 200 | 10778 |
| 9 | Grass | 9469 | 200 | 9269 |
| 10 | Red roof | 10516 | 200 | 10316 |
| 11 | Gray roof | 16911 | 200 | 16711 |
| 12 | Plastic | 3679 | 200 | 3479 |
| 13 | Bare soil | 9116 | 200 | 8916 |
| 14 | Road | 18560 | 200 | 18360 |
| 15 | Bright object | 1136 | 200 | 936 |
| 16 | Water | 75401 | 200 | 75201 |
| | Total | 257530 | 3200 | 254330 |

TABLE VII

SAMPLES' INFORMATION OF TRAINING AND TESTING FOR EACH CATEGORY FOR THE WHLK DATASET

| Class NO. | Class Name | Total | Train | Test |
|---|---|---|---|---|
| 1 | Corn | 34511 | 200 | 34311 |
| 2 | Cotton | 8374 | 200 | 8174 |
| 3 | Sesame | 3031 | 200 | 2831 |
| 4 | Broad-leaf soybean | 63212 | 200 | 63012 |
| 5 | Narrow-leaf soybean | 4151 | 200 | 3951 |
| 6 | Rice | 11854 | 200 | 11654 |
| 7 | Water | 67056 | 200 | 66856 |
| 8 | Roads and houses | 7124 | 200 | 6924 |
| 9 | Mixed weed | 5229 | 200 | 5029 |
| | Total | 204542 | 1800 | 202742 |

TABLE VIII

SAMPLES' INFORMATION OF TRAINING AND TESTING FOR EACH CATEGORY FOR THE HU DATASET

| Class NO. | Class Name | Total | Train | Test |
|---|---|---|---|---|
| 1 | Healthy Grass | 1251 | 200 | 1051 |
| 2 | Stressed Grass | 1254 | 200 | 1054 |
| 3 | Synthetic Grass | 697 | 200 | 497 |
| 4 | Tree | 1244 | 200 | 1044 |
| 5 | Soil | 1252 | 200 | 1052 |
| 6 | Water | 325 | 200 | 125 |
| 7 | Residential | 1268 | 200 | 1068 |
| 8 | Commercial | 1244 | 200 | 1044 |
| 9 | Road | 1252 | 200 | 1052 |
| 10 | Highway | 1227 | 200 | 1027 |
| 11 | Railway | 1235 | 200 | 1035 |
| 12 | Parking Lot1 | 1233 | 200 | 1033 |
| 13 | Parking Lot2 | 469 | 200 | 269 |
| 14 | Tennis Court | 428 | 200 | 228 |
| 15 | Running Track | 660 | 200 | 460 |
| | Total | 15039 | 3000 | 12039 |

and testing samples from affecting the classification results, ten different random seeds are chosen for each experiment.

### C. Parameter Influence

*1) Influence of Spatial Size:* The spatial size is an essential factor affecting the classification accuracy of the network. If the spatial size is too small, the network cannot extract sufficient spatial features, resulting in a decrease in classification accuracy. If the spatial size is too large, the network cannot extract the spatial features of small targets. In order to find a suitable spatial size, the experiments in this section test the classification accuracy of the proposed model when the spatial sizes are $9 \times 9$, $11 \times 11$, $13 \times 13$, $15 \times 15$, and $17 \times 17$. In order to ensure the accuracy of the experiments, other experimental conditions are kept the same, and the time steps of the network are set to 30. Table IX shows the effect of different spatial sizes on the classification accuracy of six HSIs' datasets.

TABLE IX

OA, AA, AND KAPPA COEFFICIENT ($K$) FOR DIFFERENT SPATIAL SIZES ON SIX HYPERSPECTRAL DATASETS

| Data Set | Spatial Size | OA (%) | AA (%) | K × 100 |
|---|---|---|---|---|
| IP | 9 | 97.81±0.60 | **99.05±0.26** | 97.41±0.71 |
| | 11 | 98.12±0.42 | 98.97±0.44 | 97.78±0.49 |
| | **13** | **98.21±0.41** | 98.90±0.87 | **97.88±0.49** |
| | 15 | 97.99±0.53 | 98.82±0.92 | 97.62±0.62 |
| | 17 | 97.97±0.48 | 98.78±0.67 | 97.60±0.57 |
| PU | 9 | 99.00±0.25 | 99.07±0.28 | 98.65±0.34 |
| | 11 | 99.19±0.34 | 99.30±0.16 | 98.91±0.45 |
| | **13** | **99.36±0.22** | **99.31±0.15** | **99.14±0.30** |
| | 15 | 99.33±0.13 | 99.29±0.08 | 99.10±0.18 |
| | 17 | 99.31±0.21 | 99.32±0.20 | 99.07±0.28 |
| SA | 9 | 98.81±0.37 | 99.53±0.14 | 98.67±0.42 |
| | 11 | 99.17±0.42 | 99.67±0.13 | 99.08±0.47 |
| | 13 | 99.36±0.25 | 99.75±0.09 | 99.28±0.28 |
| | 15 | 99.56±0.21 | **99.82±0.08** | 99.50±0.24 |
| | **17** | **99.58±0.29** | 99.82±0.11 | **99.53±0.33** |
| HU | 9 | 99.20±0.22 | 99.37±0.18 | 99.13±0.24 |
| | **11** | **99.27±0.19** | **99.67±0.13** | **99.08±0.47** |
| | 13 | 99.24±0.13 | 99.40±0.11 | 99.18±0.14 |
| | 15 | 99.16±0.21 | 99.33±0.16 | 99.09±0.23 |
| | 17 | 99.15±0.22 | 99.32±0.18 | 99.08±0.24 |
| WHHC | 9 | 94.85±0.42 | 95.05±0.30 | 93.98±0.49 |
| | 11 | 96.13±0.32 | 96.29±0.36 | 95.47±0.38 |
| | 13 | 96.59±0.32 | 96.96±0.24 | 96.01±0.38 |
| | 15 | 97.02±0.35 | 97.25±0.27 | 96.52±0.41 |
| | **17** | **97.33±0.30** | **97.61±0.35** | **96.87±0.35** |
| WHLK | 9 | 98.93±0.28 | 98.93±0.22 | 98.59±0.36 |
| | 11 | 98.95±0.23 | 98.81±0.32 | 98.63±0.30 |
| | 13 | 98.87±0.19 | 98.98±0.11 | 98.51±0.24 |
| | **15** | **99.05±0.26** | **99.05±0.09** | **98.75±0.33** |
| | 17 | 99.04±0.24 | 98.96±0.18 | 98.73±0.32 |

The experimental results show that, on the IP and PU datasets, the proposed model's OA is improved with the increase in spatial size. When the spatial size reaches 13, the OA of the two datasets is the highest, respectively, 98.12% and 99.36%. However, when the spatial size continues to increase, the OA of the proposed model decreases. On the SA and WHHC datasets, the classification accuracy continues to increase as the spatial size increases. When the spatial size is 17, the OA of the proposed model reaches the maximum, 99.58% and 97.33%, respectively. On the HU dataset, the three classification evaluation indicators, OA, AA, and $K$'s highest results, are, respectively, 99.27%, 99.67%, and 99.08%. On the

TABLE X

OA, AA, AND KAPPA COEFFICIENT ($K$) FOR DIFFERENT TIME STEPS ON SIX HYPERSPECTRAL DATASETS

| Data Set | Time-Steps | OA (%) | AA (%) | K × 100 |
|---|---|---|---|---|
| IP | 10 | 97.46±0.94 | 98.79±0.58 | 96.99±1.12 |
| | 20 | 98.04±0.68 | **99.18±0.29** | 97.68±0.80 |
| | **30** | **98.21±0.41** | 98.90±0.87 | **97.88±0.49** |
| | 40 | 98.15±0.38 | 98.98±0.69 | 97.81±0.44 |
| PU | 10 | 99.04±0.33 | 99.03±0.25 | 98.71±0.45 |
| | 20 | 99.26±0.18 | 99.25±0.17 | 99.00±0.24 |
| | 30 | 99.36±0.22 | **99.31±0.15** | 99.14±0.30 |
| | **40** | **99.38±0.16** | 99.30±0.14 | **99.16±0.21** |
| SA | 10 | 99.35±0.31 | 99.75±0.09 | 99.27±0.35 |
| | 20 | 99.42±0.38 | 99.77±0.15 | 99.35±0.43 |
| | 30 | 99.58±0.29 | 99.82±0.11 | 99.53±0.33 |
| | **40** | **99.63±0.17** | **99.84±0.08** | **99.59±0.19** |
| HU | 10 | 98.96±0.23 | 99.15±0.18 | 98.87±0.25 |
| | 20 | 99.17±0.45 | 99.34±0.36 | 99.09±0.49 |
| | **30** | **99.27±0.19** | **99.41±0.17** | **99.21±0.21** |
| | 40 | 99.19±0.24 | 99.34±0.19 | 99.12±0.27 |
| WHHC | 10 | 97.27±0.42 | 97.54±0.36 | 96.80±0.50 |
| | 20 | 97.30±0.25 | 97.56±0.22 | 96.84±0.30 |
| | 30 | 97.33±0.30 | **97.61±0.35** | 96.87±0.35 |
| | **40** | **97.34±0.18** | 97.55±0.22 | **96.88±0.21** |
| WHLK | 10 | 98.92±0.23 | 98.96±0.24 | 98.58±0.30 |
| | 20 | 98.93±0.28 | 98.94±0.20 | 98.59±0.36 |
| | **30** | **99.05±0.26** | **99.05±0.09** | **98.75±0.33** |
| | 40 | 99.00±0.45 | 99.00±0.24 | 98.68±0.58 |

WHLK dataset, when the spatial size reaches 15, the classification accuracy is the highest, and the classification evaluation indicators, OA, AA, and $K$, are 99.05%, 99.05%, and 98.75%, respectively. We found that the optimal spatial sizes of the six hyperspectral datasets (IP, PU, SA, HU, WHHC, and WHLK) are 13, 13, 17, 11, 17, and 15, respectively.

*2) Influence of Time Steps:* The spike time step is an important parameter that affects the classification accuracy of the SNN. If the time step is too tiny, the neurons cannot thoroughly learn the features of the HSIs, resulting in low classification accuracy; if the time step is too many, it will cause too many computational energy losses, which is contrary to our original intention. Therefore, we search for the optimal time step for our model through six hyperspectral datasets.

According to the previous experiments, we obtained the optimal spatial sizes of six hyperspectral datasets as 13, 13, 17, 11, 17, and 15, respectively. Therefore, for the six hyperspectral datasets, we set the time steps of the framework to 10, 20, 30, and 40, respectively. Table X shows the effect of different time steps on the classification accuracy of six HSIs' datasets. It can be seen from experiments that the framework can learn the general characteristics of HSIs over ten time steps. Over 30–40 time steps, the framework shows good classification accuracy on six datasets. Through careful consideration, we set 30 as the best time steps for our model, at which time our model can show good classification accuracy without causing too much computational consumption.

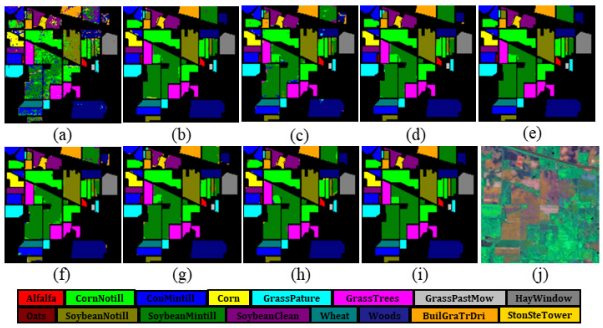



Fig. 10. Classification maps of the different methods for the IP dataset. (a) MLP. (b) SSRN. (c) DPyResNet. (d) SSDC. (e) A2S2K-ResNet. (f) NPF. (g) SSFTT. (h) Proposed. (i) Ground-truth image. (j) Three-band false color composite image.

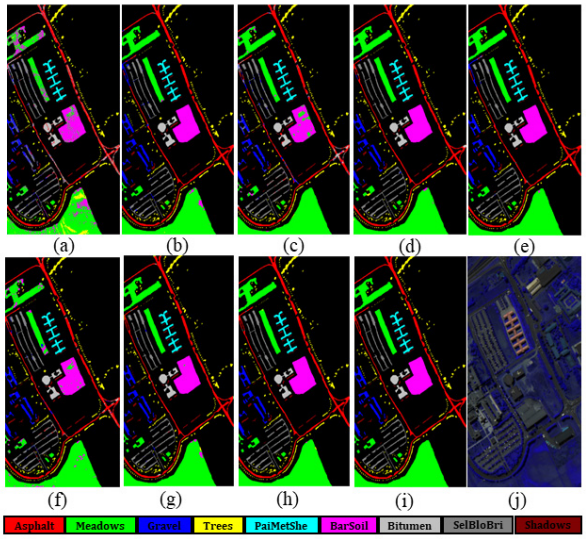



Fig. 11. Classification maps of the different methods for the PU dataset. (a) MLP. (b) SSRN. (c) DPyResNet. (d) SSDC. (e) A2S2K-ResNet. (f) NPF. (g) SSFTT. (h) Proposed. (i) Ground-truth image. (j) Three-band false color composite image.

## IV. RESULTS' DISCUSSION AND APPROACH EVALUATION

In this section, we compare the proposed model with the current mainstream models and the SNN model, SNN-SSEM, for HSI classification and estimate the energy consumption of the proposed model by calculating the number of total operations. In addition, we verify the effectiveness of our approximate derivative algorithm and model architecture.

### A. Comparison With Artificial Neural Networks

To further verify the classification effectiveness of the proposed model, we perform experiments on four benchmark hyperspectral datasets (IP, PU, SA, and HU) to compare the performance of the proposed model with six other methods, namely, multilayer perceptron (MLP), SSRN [17], DPyResNet [49], SSDC [50], A2S2K-ResNet [51], NPF [52], and SSFTT [21]. The parameters of the comparison models are basically set according to the parameters of the original paper. The parameters are set roughly as follows: the

TABLE XI
CLASSIFICATION RESULTS OF DIFFERENT METHODS FOR THE IP DATASET

| Class NO. | MLP | SSRN | DPyResNet | SSDC | A2S2K-ResNet | NPF | SSFTT | Proposed |
|---|---|---|---|---|---|---|---|---|
| 1 | 86.67 | 100.00 | 98.89 | 98.89 | 100.00 | 100.00 | 96.67 | 97.78 |
| 2 | 79.68 | 98.00 | 92.34 | 97.41 | 97.91 | 93.25 | 97.21 | 96.78 |
| 3 | 72.14 | 99.08 | 95.83 | 98.67 | 99.13 | 96.86 | 99.40 | 99.17 |
| 4 | 87.84 | 100.00 | 100.00 | 100.00 | 100.00 | 100.00 | 100.00 | 100.00 |
| 5 | 92.08 | 99.65 | 98.59 | 99.68 | 99.82 | 99.22 | 99.40 | 99.47 |
| 6 | 96.49 | 99.96 | 98.11 | 99.30 | 99.72 | 99.08 | 99.92 | 99.57 |
| 7 | 86.00 | 100.00 | 100.00 | 100.00 | 100.00 | 98.00 | 100.00 | 100.00 |
| 8 | 99.10 | 100.00 | 99.53 | 100.00 | 100.00 | 99.93 | 100.00 | 100.00 |
| 9 | 75.00 | 92.50 | 95.00 | 100.00 | 100.00 | 97.50 | 100.00 | 97.50 |
| 10 | 79.03 | 98.54 | 95.48 | 98.13 | 98.61 | 95.16 | 98.55 | 98.96 |
| 11 | 61.77 | 97.09 | 87.61 | 97.21 | 97.33 | 90.33 | 96.25 | 96.70 |
| 12 | 83.03 | 98.32 | 96.87 | 98.50 | 98.50 | 98.09 | 97.76 | 98.30 |
| 13 | 98.00 | 100.00 | 100.00 | 100.00 | 100.00 | 100.00 | 100.00 | 100.00 |
| 14 | 87.44 | 99.82 | 96.96 | 99.84 | 99.87 | 97.86 | 99.46 | 99.43 |
| 15 | 81.18 | 100.00 | 99.25 | 100.00 | 100.00 | 99.35 | 99.62 | 99.84 |
| 16 | 96.11 | 100.00 | 99.44 | 100.00 | 100.00 | 100.00 | 100.00 | 100.00 |
| OA (%) | 77.45±1.32 | 98.47±0.53 | 93.55±0.88 | 98.31±0.49 | **98.55±0.39** | 94.84±0.63 | 98.03±0.38 | 98.12±0.42 |
| AA (%) | 85.10±1.09 | 98.93±1.44 | 97.12±0.75 | 99.23±0.29 | **99.43±0.14** | 97.79±0.67 | 99.02±0.56 | 98.97±0.44 |
| K × 100 | 73.83±1.40 | 98.19±0.62 | 92.41±1.03 | 98.00±0.58 | **98.28±0.47** | 93.91±0.74 | 97.67±0.45 | 97.78±0.49 |

TABLE XII
TRAINING TIME, TEST TIME, AND OA OF SSRN, SSDC, A2S2K-RESNET, AND THE PROPOSED MODEL FOR THE THREE DATASETS

| DataSet | Model | Training Time (s) | Test Time (s) | OA (%) |
|---|---|---|---|---|
| IP | SSRN | 12634.29±27.82 | 169.72±0.44 | 98.47±0.53 |
| | SSDC | **350.12±70.48** | **4.59±8.65** | 98.31±0.49 |
| | A2S2K-ResNet | 17918.25±28.66 | 246.49±0.75 | **98.55±0.39** |
| | Ours | 2022.60±43.56 | 42.98±2.20 | 98.12±0.42 |
| PU | SSRN | 9206.75±25.14 | 883.72±1.99 | 98.95±0.44 |
| | SSDC | **258.63±69.45** | **11.07±8.32** | 98.82±0.41 |
| | A2S2K-ResNet | 11326.33±33.72 | 1095.39±1.67 | 99.18±0.36 |
| | Ours | 1535.67±50.33 | 230.19±3.22 | **99.19±0.34** |
| SA | SSRN | 15678.74±22.34 | 1107.28±1.36 | 97.88±0.67 |
| | SSDC | **430.60±39.33** | **14.65±8.34** | 97.38±0.47 |
| | A2S2K-ResNet | 22419.77±40.99 | 1622.33±3.99 | 97.84±0.45 |
| | Ours | 2605.56±51.15 | 285.48±2.95 | **99.17±0.42** |

epochs of MLP are set to 100, the epochs of NPF are set to 500, and the training of other comparable models uses 200 epochs. In addition, PyResNet uses the SGD optimizer, SSRN uses the RMSProp [53] optimizer, and other comparable models use the Adam [54] optimizer. For the objectivity of the comparative experiment, the spatial size of the input HSIs is set to 11, each group of experiments is performed ten times, and the average values and standard deviation of the experimental results are taken as the final result. Most code of the comparison models is available on these websites (https://github.com/suvojit-0x55aa/A2S2K-ResNet and https://github.com/zgr6010/HSI_SSFTT).

*1) Classification Results and Classification Maps on IP Dataset:* The classification results of the models are shown in Table XI, and the classification maps are shown in Fig. 10. The classification performance of the proposed model on the IP dataset is better than MLP, DPyResNet, NPF, and SSFTT. However, the classification performance is not as good as the models: SSRN, SSDC, and A2S2K-ResNet. The evaluation

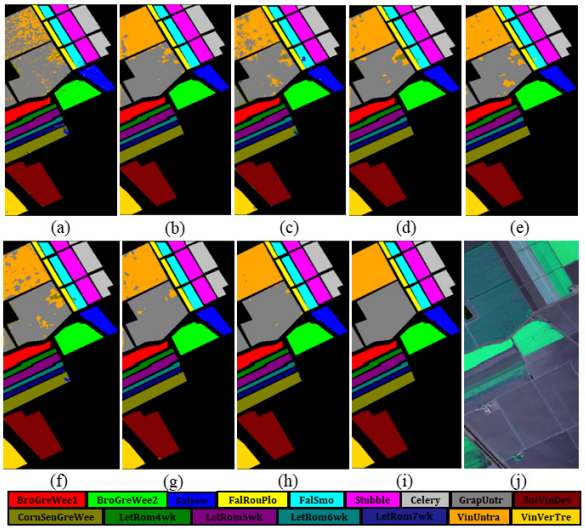



Fig. 12. Classification maps of the different methods for the SV dataset. (a) MLP. (b) SSRN. (c) DPyResNet. (d) SSDC. (e) A2S2K-ResNet. (f) NPF. (g) SSFTT. (h) Proposed. (i) Ground-truth image. (j) Three-band false color composite image.

index, OA, of the proposed model is about 0.35%, 0.19%, and 0.43% lower than the above three models, respectively. To further compare the classification performance of four models (SSRN, SSDC, A2S2K-ResNet, and the proposed model), we recorded the average training and test time of four models in ten-time experiments under the same hardware conditions. However, due to deployment issues, we can only compare the computation time of these models on nonneuromorphic computers (or von Neuman computers) [28]. The average training time and testing time are shown in Table XII. From the experimental results, it can be seen that the training time on the nonneuromorphic computer of our model is about six times faster than that of SSRN and about eight times faster

TABLE XIII
CLASSIFICATION RESULTS OF DIFFERENT METHODS FOR THE PU DATASET

| Class NO. | MLP | SSRN | DPyResNet | SSDC | A2S2K-ResNet | NPF | SSFTT | Proposed |
|---|---|---|---|---|---|---|---|---|
| 1 | 73.71 | 98.50 | 89.30 | 98.33 | 98.54 | 94.59 | 98.01 | 99.45 |
| 2 | 73.71 | 98.78 | 95.05 | 98.71 | 99.21 | 92.88 | 98.93 | 99.13 |
| 3 | 74.18 | 98.62 | 89.62 | 98.76 | 98.47 | 95.01 | 98.28 | 98.92 |
| 4 | 94.00 | 99.40 | 97.97 | 97.96 | 99.61 | 97.86 | 98.09 | 98.15 |
| 5 | 99.42 | 99.97 | 99.85 | 99.87 | 100.00 | 99.91 | 100.00 | 99.91 |
| 6 | 83.53 | 99.92 | 94.64 | 99.83 | 99.92 | 93.85 | 99.43 | 99.77 |
| 7 | 90.72 | 99.68 | 91.65 | 99.99 | 99.88 | 97.60 | 99.91 | 99.94 |
| 8 | 78.67 | 98.36 | 91.24 | 98.68 | 98.62 | 95.46 | 97.19 | 98.60 |
| 9 | 99.61 | 99.95 | 99.75 | 99.92 | 100.00 | 99.68 | 99.81 | 99.85 |
| OA (%) | 79.95±5.53 | 98.95±0.44 | 93.85±0.77 | 98.82±0.41 | 99.18±0.36 | 94.38±0.85 | 98.68±0.55 | **99.19±0.34** |
| AA (%) | 85.67±3.82 | 99.24±0.29 | 94.34±0.56 | 99.11±0.16 | **99.36±0.35** | 96.32±0.32 | 98.85±0.34 | 99.30±0.16 |
| K × 100 | 74.26±6.56 | 98.59±0.59 | 91.80±0.99 | 98.41±0.55 | 98.90±0.49 | 92.53±1.10 | 98.20±0.73 | **98.91±0.45** |

TABLE XIV
CLASSIFICATION RESULTS OF DIFFERENT METHODS FOR THE SA DATASET

| Class NO. | MLP | SSRN | DPyResNet | SSDC | A2S2K-ResNet | NPF | SSFTT | Proposed |
|---|---|---|---|---|---|---|---|---|
| 1 | 98.38 | 100.00 | 99.67 | 100.00 | 100.00 | 99.98 | 100.00 | 100.00 |
| 2 | 99.31 | 100.00 | 99.82 | 100.00 | 99.99 | 99.99 | 99.95 | 99.98 |
| 3 | 99.01 | 99.98 | 99.57 | 100.00 | 99.98 | 99.80 | 100.00 | 99.98 |
| 4 | 99.43 | 99.97 | 99.68 | 99.93 | 99.97 | 99.78 | 99.84 | 99.93 |
| 5 | 97.89 | 99.42 | 97.57 | 99.80 | 99.50 | 99.02 | 99.66 | 99.63 |
| 6 | 99.66 | 100.00 | 99.98 | 100.00 | 99.98 | 100.00 | 100.00 | 100.00 |
| 7 | 99.60 | 100.00 | 99.59 | 100.00 | 99.99 | 99.88 | 99.97 | 99.96 |
| 8 | 76.03 | 92.80 | 86.34 | 90.71 | 93.50 | 88.43 | 96.09 | 97.29 |
| 9 | 99.39 | 100.00 | 99.27 | 100.00 | 99.98 | 99.85 | 99.97 | 99.99 |
| 10 | 91.68 | 99.71 | 97.62 | 99.61 | 99.75 | 97.78 | 99.55 | 99.71 |
| 11 | 96.20 | 100.00 | 99.22 | 99.95 | 99.95 | 99.50 | 99.93 | 99.97 |
| 12 | 99.33 | 100.00 | 99.92 | 100.00 | 100.00 | 99.99 | 99.99 | 99.98 |
| 13 | 98.69 | 100.00 | 99.59 | 100.00 | 100.00 | 99.78 | 100.00 | 100.00 |
| 14 | 95.85 | 99.93 | 99.44 | 99.97 | 99.95 | 99.75 | 99.92 | 99.97 |
| 15 | 66.14 | 96.35 | 88.75 | 95.94 | 94.97 | 91.03 | 97.66 | 98.70 |
| 16 | 98.53 | 99.89 | 99.11 | 99.83 | 99.88 | 99.39 | 99.66 | 99.69 |
| OA (%) | 88.98±1.75 | 97.88±0.67 | 94.99±0.55 | 97.38±0.47 | 97.84±0.45 | 95.99±0.45 | 98.75±0.58 | **99.17±0.42** |
| AA (%) | 94.69±0.49 | 99.25±0.22 | 97.82±0.34 | 99.11±0.16 | 99.21±0.14 | 98.37±0.24 | 99.51±0.19 | **99.67±0.13** |
| K × 100 | 87.69±1.90 | 97.62±0.75 | 94.39±0.62 | 97.07±0.53 | 97.58±0.50 | 95.51±0.50 | 98.61±0.65 | **99.08±0.47** |

TABLE XV
CLASSIFICATION RESULTS OF DIFFERENT METHODS FOR THE HU DATASET

| Class NO. | MLP | SSRN | DPyResNet | SSDC | A2S2K-ResNet | NPF | SSFTT | Proposed |
|---|---|---|---|---|---|---|---|---|
| 1 | 96.22 | 98.84 | 96.18 | 98.90 | 99.01 | 97.78 | 98.85 | 99.01 |
| 2 | 95.76 | 99.64 | 97.40 | 98.64 | 99.59 | 99.18 | 99.21 | 99.54 |
| 3 | 99.56 | 99.98 | 99.78 | 99.76 | 99.94 | 99.74 | 99.88 | 99.66 |
| 4 | 92.75 | 98.40 | 94.54 | 97.16 | 99.06 | 97.35 | 98.70 | 99.63 |
| 5 | 97.22 | 100.00 | 98.38 | 99.80 | 99.99 | 99.27 | 99.99 | 99.91 |
| 6 | 98.88 | 100.00 | 99.04 | 100.00 | 100.00 | 98.56 | 100.00 | 100.00 |
| 7 | 86.84 | 98.94 | 95.23 | 98.04 | 98.49 | 94.78 | 98.51 | 98.60 |
| 8 | 73.50 | 96.05 | 90.44 | 96.12 | 96.67 | 93.25 | 98.31 | 98.03 |
| 9 | 75.34 | 97.70 | 94.02 | 96.04 | 97.44 | 92.76 | 98.38 | 98.47 |
| 10 | 82.95 | 99.88 | 95.79 | 99.72 | 99.78 | 95.85 | 99.85 | 99.91 |
| 11 | 75.96 | 98.84 | 95.76 | 98.42 | 98.56 | 95.56 | 99.38 | 99.76 |
| 12 | 78.17 | 98.20 | 97.45 | 98.76 | 98.70 | 96.96 | 98.59 | 99.09 |
| 13 | 70.22 | 99.63 | 95.43 | 98.40 | 99.59 | 98.40 | 99.67 | 99.55 |
| 14 | 99.87 | 100.00 | 99.91 | 100.00 | 100.00 | 100.00 | 99.96 | 100.00 |
| 15 | 98.87 | 100.00 | 99.87 | 100.00 | 100.00 | 99.78 | 100.00 | 100.00 |
| OA (%) | 86.66±1.10 | 98.82±0.28 | 95.98±0.61 | 98.35±0.36 | 98.88±0.24 | 96.69±0.42 | 99.10±0.26 | **99.27±0.19** |
| AA (%) | 88.14±0.70 | 99.07±0.22 | 96.62±0.49 | 98.65±0.31 | 99.12±0.21 | 97.28±0.35 | 99.29±0.20 | **99.41±0.17** |
| K × 100 | 85.52±1.19 | 98.71±0.30 | 95.63±0.66 | 98.21±0.39 | 98.79±0.26 | 96.40±0.46 | 99.02±0.29 | **99.21±0.21** |

than that of A2S2K, but the training time of the SSDC model is the shortest among the four models. However, the classification model architecture used in this article is SNN. For the SNN, it needs to be deployed on neuromorphic hardware to take

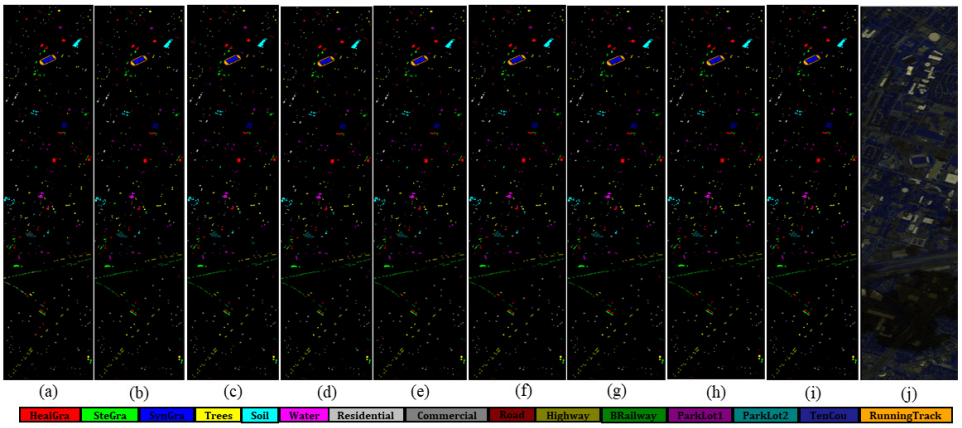



Fig. 13. Classification maps of the different methods for the HU dataset. (a) MLP. (b) SSRN. (c) DPyResNet. (d) SSDC. (e) A2S2K-ResNet. (f) NPF. (g) SSFTT. (h) Proposed. (i) Ground-truth image. (j) Three-band false color composite image.

advantage of its low energy consumption and fast computing [27], [28], [31]. In addition, the proposed model is a low-energy model. The details of the energy consumption analysis are given in Section IV-C. The computational energy consumption required for the classification of the proposed model provides unlimited possibilities for its future practicality.

*2) Classification Results and Classification Maps on PU, SA, and HU Datasets:* The classification results of the models on the PU, SA, and HU datasets are shown in Tables XIII–XV, and the classification maps are shown in Figs. 11–13. It can be seen through experiments that the evaluation indicators (OA, AA, and K) of the proposed model on the PU, SA, and HU datasets have basically achieved the highest results among these models. Compared with other models, the classification advantage of the SA and HU datasets is more prominent. The OA of the proposed model on the SA dataset is about 1.29%, 1.79%, and 1.33%, higher than that of SSRN, SSDC, and A2S2K-ResNet, respectively. The OA of the proposed model on the HU dataset is about 0.45%, 0.92%, 0.39%, and 0.17%, higher than that of SSRN, SSDC, A2S2K-ResNet, and SSFTT, respectively. In addition, the classification result of the proposed model on the eighth category of ground objects in the SA dataset is much better than that of several other models.

### *B. Comparison With Spiking Neural Network*

In order to further verify the advantages of the framework on time steps, we compare the classification effects of the proposed model with SNN-SSEM [44] over five, ten, 30, 80, and 100 time steps on IP, PU, SA, and HU hyperspectral datasets. The experimental parameter spatial size of the SNN-SSEM on IP, PU, and SA datasets takes the optimal values of 15, 17, and 17 in the original paper, respectively. The experimental parameter spatial size of the SNN model in this article on IP, PU, and SA datasets is set to 13, 13, and 17, respectively. Since SNN-SSEM does not test the model's classification performance on the HU dataset, the spatial size of SNN-SSEM on the HU dataset takes 11 as the proposed model. Three evaluation indicators (OA, AA, and $K$) measure the classification results. The experimental results are shown in Table XVI.

The spiking time step is an important metric to measure the SNN model. The fewer the time steps, the higher the accuracy, indicating that the SNN has a strong learning ability. It can be seen from the experimental results that the proposed model can learn the general characteristics of the HSIs over five time steps. In comparison, SNN-SSEM needs about 80 time steps to learn the general characteristics of the images. SNN-SSEM can achieve higher classification accuracy over 100 time steps, but the proposed model basically reaches the peak of the classification accuracy over about 30–40 time steps. The experimental results show that the proposed model for classifying HSIs can significantly reduce the number of time steps to achieve high classification accuracy and saves energy calculations compared with SNN-SSEM. To further explore the time required for the two models to achieve classification on different time steps, we compare the training and testing times of the two models on the PU and HU datasets when the time steps are 10, 30, and 100. The experimental results are shown in Table XVII. The experimental results show that our model requires more training time and testing time than SNN-SSEM at the same time step on PU and HU datasets. However, compared with the optimal time step, the training time required by the proposed model is about three times less than that of SNN-SSEM, and the required testing time is also less than that of SNN-SSEM on PU and HU datasets.

In addition, the proposed model's design is based on LIF neurons, while the design of SNN-SSEM is based on integrate-and-fire (IF) neurons. Compared with IF neurons, LIF neurons add the leakage characteristic, and the leakage characteristic belongs to an essential characteristic of biological neurons. The leakage characteristic of neurons in the network will lead to the fact that the extraction ability of HSIs' features is not as good as that of IF neurons under more time steps (such as 80 and 100). However, the design principle of the model in this article is more suitable for the brain-inspired than the design principle of SNN-SSEM.

### *C. Network Computing Energy Consumption Estimation*

Recently, some researchers have constructed a simplified brain-inspired SNN architecture through superconducting hardware materials or brain-inspired chips to calculate the

TABLE XVI
CLASSIFICATION RESULTS OF DIFFERENT TIME STEPS ON SNN-SSEM AND THE PROPOSED MODEL ON FOUR HYPERSPECTRAL DATASETS

| | | SNN-SSEM | | | Proposed | | |
|---|---|---|---|---|---|---|---|
| Data Set | Time-Steps | OA (%) | AA (%) | K × 100 | OA (%) | AA (%) | K × 100 |
| IP | 5 | 1.87±5.25 | 7.09±2.51 | 0.63±1.90 | **97.22±0.74** | **98.35±0.74** | **96.71±0.87** |
| | 10 | 36.24±7.20 | 36.32±8.28 | 29.22±7.06 | **97.46±0.94** | **98.79±0.58** | **96.99±1.12** |
| | 30 | 91.59±12.80 | 92.46±14.35 | 90.32±14.56 | **98.21±0.41** | **98.90±0.87** | **97.88±0.49** |
| | 80 | **98.79±0.26** | **99.48±0.26** | **98.57±0.31** | 98.00±0.58 | 99.04±0.23 | 97.63±0.68 |
| | 100 | **98.88±0.29** | **99.59±0.10** | **98.67±0.35** | 97.79±0.64 | 98.77±0.66 | 97.38±0.75 |
| PU | 5 | 15.86±0.49 | 12.68±4.72 | 1.04±3.13 | **98.77±0.43** | **98.79±0.30** | **98.35±0.57** |
| | 10 | 51.63±13.71 | 46.95±11.07 | 38.58±12.00 | **99.04±0.33** | **99.03±0.25** | **98.71±0.45** |
| | 30 | 93.11±11.68 | 94.60±8.93 | 91.08±14.96 | **99.36±0.22** | **99.31±0.15** | **99.14±0.30** |
| | 80 | **99.43±0.41** | **99.60±0.24** | **99.24±0.55** | 99.38±0.17 | 99.37±0.16 | 99.17±0.22 |
| | 100 | **99.62±0.25** | **99.73±0.09** | **99.49±0.33** | 99.24±0.34 | 99.21±0.25 | 98.98±0.45 |
| SA | 5 | 5.26±3.45 | 7.45±2.43 | 1.37±2.83 | **99.36±0.39** | **99.75±0.12** | **99.28±0.44** |
| | 10 | 36.59±9.15 | 43.08±5.50 | 32.25±8.65 | **99.35±0.31** | **99.75±0.09** | **99.27±0.35** |
| | 30 | 90.61±14.39 | 93.87±10.56 | 89.65±15.79 | **99.58±0.29** | **99.82±0.11** | **99.53±0.33** |
| | 80 | 99.66±0.21 | 99.87±0.07 | 99.62±0.23 | **99.68±0.12** | **99.85±0.06** | **99.64±0.14** |
| | 100 | **99.70±0.17** | **99.88±0.07** | **99.66±0.19** | 99.56±0.30 | 99.80±0.14 | 99.51±0.33 |
| HU | 5 | 8.36±1.14 | 7.88±3.63 | 0.31±0.92 | **98.66±0.30** | **98.90±0.28** | **98.54±0.32** |
| | 10 | 30.97±7.27 | 39.43±7.42 | 26.29±7.79 | **98.96±0.23** | **99.15±0.18** | **98.87±0.25** |
| | 30 | 91.28±15.94 | 92.73±13.37 | 90.61±17.16 | **99.27±0.19** | **99.41±0.17** | **99.21±0.21** |
| | 80 | **99.44±0.22** | **99.56±0.18** | **99.39±0.24** | 99.27±0.24 | 99.41±0.20 | 99.20±0.26 |
| | 100 | **99.46±0.15** | **99.56±0.14** | **99.49±0.33** | 99.10±0.35 | 99.26±0.30 | 99.03±0.38 |

TABLE XVII
TRAINING TIME, TEST TIME, AND OA OF DIFFERENT TIME STEPS ON SNN-SSEM AND THE PROPOSED MODEL ON PU AND HU HYPERSPECTRAL DATASETS

| | | SNN-SSEM | | | Proposed | | |
|---|---|---|---|---|---|---|---|
| Data Set | Time-Steps | OA (%) | Training Time (s) | Test Time (s) | OA (%) | Training Time (s) | Test Time (s) |
| PU | 10 | 51.63±13.71 | 527.81±25.85 | 35.70±0.28 | 99.04±0.33 | 590.01±55.02 | 93.84±5.86 |
| | 30 | 93.11±11.68 | 1533.62±26.42 | 96.11±1.20 | 99.36±0.22 | **1643.13±55.75** | **260.43±7.43** |
| | 100 | 99.62±0.25 | **5151.93±40.20** | **307.72±1.22** | 99.24±0.34 | 5459.21±77.35 | 872.61±11.39 |
| HU | 10 | 30.97±7.27 | 375.12±1.60 | 7.03±0.08 | 98.96±0.23 | 306.79±5.83 | 6.75±0.13 |
| | 30 | 91.28±15.94 | 1071.44±3.04 | 19.02±0.13 | 99.27±0.19 | **994.09±3.55** | **18.76±0.09** |
| | 100 | 99.46±0.15 | **2813.33±26.07** | **67.62±1.18** | 99.10±0.35 | 2745.59±17.12 | 54.44±0.30 |

TABLE XVIII
CLASSIFICATION RESULTS OF THE APPROXIMATE DERIVATIVE ALGORITHM APPLIED TO SNN ON SIX HYPERSPERCTRAL DATASETS

| | IP | PU | SA | HU | WHHC | WHLK |
|---|---|---|---|---|---|---|
| OA (%) | 98.11±0.35 | 99.12±0.36 | 99.25±0.35 | 99.12±0.18 | 96.25±0.23 | 98.91±0.29 |
| AA (%) | 99.18±0.21 | 99.11±0.32 | 99.70±0.13 | 99.29±0.13 | 96.81±0.24 | 98.99±0.17 |
| K × 100 | 97.76±0.41 | 98.81±0.48 | 99.16±0.39 | 99.04±0.20 | 95.93±0.27 | 98.57±0.38 |

TABLE XIX
IMPLEMENTATION DETAILS OF MODEL_SUB1 AND MODEL_SUB2

| Model_Sub1 | | Model_Sub1 | |
|---|---|---|---|
| Layer | Input Shape | Layer | Input Shape |
| Spikes Encoding Layer | (13×13,30) | Spikes Encoding Layer | (13×13,30) |
| SConv Layer(K=3, S=1) | (13×13,64) | DConv Layer(K=3, S=1) | (13×13,32) |
| SConv Layer(K=3, S =1) | (13×13,64) | PConv Layer(K=1, S=1) | (13×13,32) |
| SConv Layer(K=3, S =1) | (13×13,64) | SConv Layer(K=3, S=1) | (13×13,32) |
| Average Pooling Layer(K=2) | (13×13,128) | Average Pooling Layer(K=2) | (13×13,128) |
| SConv Layer(K=3, S=1) | (6×6,128) | DConv Layer(K= 3, S=1) | (6×6,128) |
| SConv Layer(K=3, S=1) | (6×6,128) | PConv Layer(K=1, S=1) | (6×6,128) |
| SConv Layer(K=3, S=1) | (6×6,128) | SConv Layer(K=3, S=1) | (6×6,128) |
| Average Pooling Layer (K=2) | (6×6,256) | Average Pooling Layer(K=2) | (6×6,256) |
| Spikes Decoding Layer | (3×3×256, Class) | Spikes Decoding Layer | (3×3×256, Class) |

actual energy consumption value [29], [30], [31]. Since our research does not include hardware aspects, the SNN model constructed in this article cannot calculate the actual energy consumption value. However, some of the current literature on SNN roughly estimate the model's energy consumption by calculating the number of network operations [38], [44].

This article records the average number of spiking neurons activated in each layer, estimates the number of total operations in the model, and compares the computational complexity of the proposed model and CNN with the same

TABLE XX
CLASSIFICATION RESULTS OF MODEL_SUB1 AND MODEL_SUB2 ON SIX HYPERSPERCTRAL DATASETS

| | | IP | PU | SA | HU | WHHC | WHLK |
|---|---|---|---|---|---|---|---|
| Model_Sub1 | OA (%) | 97.83±0.78 | 99.20±0.24 | 99.07±0.43 | 99.19±0.17 | 96.13±0.55 | **99.06±0.23** |
| | AA (%) | 98.66±0.80 | 99.14±0.28 | 99.66±0.13 | 99.34±0.14 | 96.41±0.37 | **99.08±0.13** |
| | K × 100 | 97.43±0.92 | 98.92±0.32 | 98.97±0.48 | 99.12±0.19 | 95.47±0.64 | **98.76±0.30** |
| Model_Sub2 | OA (%) | 97.90±0.48 | 98.74±0.51 | 99.01±0.57 | 98.65±0.38 | 95.19±0.58 | 98.86±0.26 |
| | AA (%) | 99.08±0.28 | 98.92±0.32 | 99.62±0.17 | 98.89±0.33 | 95.54±0.39 | 98.92±0.18 |
| | K × 100 | 97.51±0.57 | 98.31±0.68 | 98.89±0.64 | 98.54±0.41 | 94.38±0.68 | 98.50±0.34 |
| Proposed | OA (%) | **98.21±0.41** | **99.36±0.22** | **99.36±0.25** | **99.24±0.13** | **96.59±0.32** | 98.87±0.19 |
| | AA (%) | **98.90±0.87** | **99.31±0.15** | **99.75±0.09** | **99.40±0.11** | **96.96±0.24** | 98.98±0.11 |
| | K × 100 | **97.88±0.49** | **99.14±0.30** | **99.28±0.28** | **99.18±0.14** | **96.01±0.38** | 98.51±0.24 |

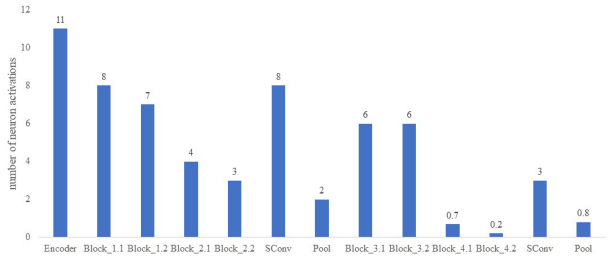



Fig. 14. Average number of neurons activated in each layer of the proposed model on the PU dataset (Block_1.1, Block_1.2, Block_3.1, and Block_3.2 represent spiking standard convolution; Block_2.1 and Block_4.1 represent spiking depthwise convolution; and Block_2.2 and Block_4.2 represent spiking pointwise convolution).

architecture. Assuming that an SNN has $l$ layers, each layer has $s_l$ synaptic connections, the average activation rate of a synapse in each layer is $a_l$, and the time step is timesteps. The accumulate computation (AC) operation of an SNN is $\sum_l s_l \times a_l \times$ time steps, and then, $\sum_l s_l \times a_l$ is the multiply AC (MAC) operation of a CNN with the same structure. From this [55], it can be seen that the energy consumption of MAC operation is usually an order of magnitude higher than that of the AC operation. For example, using 45-nm technology, the energy consumed by the 32-bit floating-point MAC operation is about 4.6 PJ, and the AC operation is about 0.9 PJ. The energy consumed in the 32-bit integer MAC operation is about 3.2 PJ, and in the AC operation, it is about 0.1 PJ. Lin *et al.* [56] state that 32-bit floating-point computations can be replaced by fixed-point computations using integer MAC and AC units with little to no penalty.

On the PU dataset, we record the average number of activations of neurons in each layer, as shown in Fig. 14. According to the calculation method mentioned above, the proposed model's computational energy efficiency is less than ten times that of an equivalent architecture CNN.

### D. Algorithm Evaluation

In order to further verify the effectiveness of the approximate derivative algorithm proposed in this article, we apply this algorithm to the SNN-SSEM [44] model, which is used to classify HSI. Then, we construct a classification model SNN with the same architecture as SNN-SSEM.

The experiments are verified on six hyperspectral datasets. The spatial size is set to 13, the time steps are set to 30, and each group of experiments is performed ten times. The experimental results are shown in Table XVIII. Through experiments, it can be seen that the approximate derivative algorithm proposed in this article has achieved good classification accuracy in the same architecture as the SNN-SSEM. The algorithm proposed in this article can realize the backpropagation of spatiotemporal spikes in the network to complete the HSI classification.

### E. Model Evaluation

In order to verify the classification validity of the proposed model, we disassemble the two branches of the proposed model and construct two submodules: Model_Sub1 and Model_Sub2. The specific framework of the submodules is shown in Table XIX, where SConv represents spiking standard convolution, DConv represents spiking depthwise convolution, PConv represents spiking pointwise convolution, $K$ represents the kernel size, and $S$ represents the stride. We test the classification performance of the two submodules on six HSI datasets. The experimental parameters' spatial size is set to 13, the time steps are set to 30, and the experiments are performed ten times. The experimental results are shown in Table XX. Through experiments, we know that the classification accuracy of the two submodules on the IP, PU, SA, HU, and WHHC datasets cannot exceed the model architecture proposed in this article, which verifies that the proposed model has certain advantages in HSI classification. However, the classification accuracy of Model_Sub1 on the WHLK dataset exceeds that of the proposed method. Based on the current related research, we have two reasonable conjectures for this result. The first is that Model_Sub1 is composed of spiking standard convolution, in which feature extraction capability is better than spiking depthwise separable convolution. Some hyperspectral features are extracted in the proposed method through spiking depthwise separable convolution. The second is that the encoding method of our model adopts direct encoding, and the direct encoding method will be affected by noise to

a certain extent [48]. However, overall, the model still has certain advantages.

## V. Conclusion

In order to promote the combination of brain-inspired computing (neuromorphic computing) and hyperspectral remote sensing, this article constructs an SNN based on LIF neurons and direct coding. The model architecture introduces spiking standard convolution and spiking depthwise separable convolution and realizes the propagation of the spatiotemporal spikes in LIF neurons through the approximate derivative algorithm. On six hyperspectral datasets, we verify the effectiveness of the model framework and algorithm, and determine the optimal classification parameters of the proposed model. Through a large number of objective and comparative experiments, it is verified that the proposed model has the characteristics of high precision and low energy consumption. In addition, we record the number of activations of neurons in each layer of the model on the PU dataset. By comparing the number of calculation operations, we found that the SNN model architecture proposed in this article can save energy about ten times compared with the CNN model architecture under the same architecture. The energy consumption estimation provides experimental support for the deployment of the brain-inspired model in edge computing devices. Currently, there is no recognized supervised learning algorithm for SNN, but, with the continuous development of brain-inspired computing, the supervised learning algorithm of SNN has a lot of room for improvement. Furthermore, with the development of neuromorphic hardware foundations and neuromorphic chips, in the future, SNNs can be deployed on neuromorphic computers to achieve real high-performance computing and can also be used to classify HSIs through neuromorphic computers.

## References


[1] W. Li, Q. Du, and B. Zhang, "Combined sparse and collaborative representation for hyperspectral target detection," *Pattern Recognit.*, vol. 48, no. 12, pp. 3904–3916, 2015.

[2] X. Yang and Y. Yu, "Estimating soil salinity under various moisture conditions: An experimental study," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 5, pp. 2525–2533, May 2017.

[3] D. Hong, N. Yokoya, J. Chanussot, and X. X. Zhu, "An augmented linear mixing model to address spectral variability for hyperspectral unmixing," *IEEE Trans. Image Process.*, vol. 28, no. 4, pp. 1923–1938, Apr. 2018.

[4] W. Xie, T. Jiang, Y. Li, X. Jia, and J. Lei, "Structure tensor and guided filtering-based algorithm for hyperspectral anomaly detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 7, pp. 4218–4230, Jul. 2019.

[5] M. Shimoni, R. Haelterman, and C. Perneel, "Hypersectral imaging for military and security applications: Combining myriad processing and sensing techniques," *IEEE Geosci. Remote Sens. Mag.*, vol. 7, no. 2, pp. 101–117, Jun. 2019.

[6] D. Hong, J. Yao, D. Meng, Z. Xu, and J. Chanussot, "Multimodal GANs: Toward crossmodal hyperspectral–multispectral image segmentation," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 6, pp. 5103–5113, Jun. 2021.

[7] Q. Zhu *et al.*, "A global context-aware and batch-independent network for road extraction from VHR satellite imagery," *ISPRS J. Photogramm. Remote Sens.*, vol. 175, pp. 353–365, May 2021.

[8] Q. Zhu *et al.*, "Land-use/land-cover change detection based on a Siamese global learning framework for high spatial resolution remote sensing imagery," *ISPRS J. Photogramm. Remote Sens.*, vol. 184, pp. 63–78, Feb. 2022.

[9] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004.

[10] Z. Zhu, S. Jia, S. He, Y. Sun, Z. Ji, and L. Shen, "Three-dimensional Gabor feature extraction for hyperspectral imagery classification using a memetic framework," *Inf. Sci.*, vol. 298, pp. 274–287, Mar. 2015.

[11] M. Belgiu and L. Drăgut, "Random forest in remote sensing: A review of applications and future directions," *ISPRS J. Photogramm. Remote Sens.*, vol. 114, pp. 24–31, Apr. 2016.

[12] G. Licciardi, P. R. Marpu, J. Chanussot, and J. A. Benediktsson, "Linear versus nonlinear PCA for the classification of hyperspectral data based on the extended morphological profiles," *IEEE Geosci. Remote Sens. Lett.*, vol. 9, no. 3, pp. 447–451, May 2011.

[13] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.

[14] W. Hu, Y. Huang, L. Wei, F. Zhang, and H. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, pp. 1–12, Jan. 2015.

[15] K. Makantasis, K. Karantzalos, A. Doulamis, and N. Doulamis, "Deep supervised learning for hyperspectral data classification through convolutional neural networks," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2015, pp. 4959–4962.

[16] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3-D–2-D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 277–281, Feb. 2019.

[17] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Oct. 2017.

[18] H. Liu, W. Li, X.-G. Xia, M. Zhang, C.-Z. Gao, and R. Tao, "Central attention network for hyperspectral imagery classification," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Mar. 10, 2022, doi: 10.1109/TNNLS.2022.3155114.

[19] D. Hong, L. Gao, J. Yao, B. Zhang, A. Plaza, and J. Chanussot, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5966–5978, Jul. 2020.

[20] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon *et al.*, Eds., vol. 30. Red Hook, NY, USA: Curran Associates, 2017. [Online]. Available: https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[21] L. Sun, G. Zhao, Y. Zheng, and Z. Wu, "Spectral–spatial feature tokenization transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–14, 2022.

[22] D. Hong *et al.*, "SpectralFormer: Rethinking hyperspectral image classification with transformers," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–15, 2021.

[23] Z. Zheng, Y. Zhong, A. Ma, and L. Zhang, "FPGA: Fast patch-free global learning framework for fully end-to-end hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 8, pp. 5612–5626, Aug. 2020.

[24] Q. Zhu *et al.*, "A spectral–spatial-dependent global learning framework for insufficient and imbalanced hyperspectral image classification," *IEEE Trans. Cybern.*, early access, May 25, 2021, doi: 10.1109/TCYB.2021.3070577.

[25] L. Dang, P. Pang, and J. Lee, "Depth-wise separable convolution neural network with residual connection for hyperspectral image classification," *Remote Sens.*, vol. 12, no. 20, p. 3408, Oct. 2020.

[26] B. Cui, X.-M. Dong, Q. Zhan, J. Peng, and W. Sun, "LiteDepthwiseNet: A lightweight network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–15, 2021.

[27] K. Roy, A. Jaiswal, and P. Panda, "Towards spike-based machine intelligence with neuromorphic computing," *Nature*, vol. 575, no. 7784, pp. 607–617, Nov. 2019.

[28] C. D. Schuman, S. R. Kulkarni, M. Parsa, J. P. Mitchell, P. Date, and B. Kay, "Opportunities for neuromorphic computing algorithms and applications," *Nature Comput. Sci.*, vol. 2, no. 1, pp. 10–19, Jan. 2022.

[29] S. Yang, Z. Kuang, B. Deng, and J. Wang, "Reconstruction of brain-inspired visual spiking neural network on bicoss," in *Proc. IEEE 3rd Int. Conf. Frontiers Technol. Inf. Comput. (ICFTIC)*, Nov. 2021, pp. 612–616.

[30] H. Zhang, C. Gang, C. Xu, G. Gong, and H. Lu, "Brain-inspired spiking neural network using superconducting devices," *IEEE Trans. Emerg. Topics Comput. Intell.*, early access, Jul. 2, 2021, doi: 10.1109/TETCI.2021.3089328.

[31] Y. Wu *et al.*, "Brain-inspired global-local learning incorporated with neuromorphic computing," *Nature Commun.*, vol. 13, no. 1, pp. 1–14, Dec. 2022.

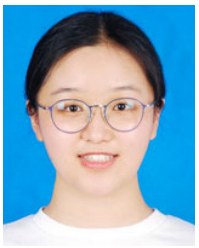
[32] A. Sengupta, Y. Ye, R. Wang, C. Liu, and K. Roy, "Going deeper in spiking neural networks: VGG and residual architectures," *Frontiers Neurosci.*, vol. 13, p. 95, Mar. 2019.
[33] B. Han, G. Srinivasan, and K. Roy, "RMP-SNN: Residual membrane potential neuron for enabling deeper high-accuracy and low-latency spiking neural network," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2020, pp. 13558–13567.
[34] Y. Li, S. Deng, X. Dong, R. Gong, and S. Gu, "A free lunch from ANN: Towards efficient, accurate spiking neural networks calibration," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 6316–6325.
[35] Z. Yan, J. Zhou, and W.-F. Wong, "Near lossless transfer learning for spiking neural networks," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 12, 2021, pp. 10577–10584.
[36] Y. Wu, L. Deng, G. Li, J. Zhu, and L. Shi, "Spatio-temporal backpropagation for training high-performance spiking neural networks," 2017, *arXiv:1706.02609*.
[37] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks," *IEEE Signal Process. Mag.*, vol. 36, no. 6, pp. 51–63, Nov. 2019.
[38] C. Lee, S. S. Sarwar, P. Panda, G. Srinivasan, and K. Roy, "Enabling spike-based backpropagation for training deep neural network architectures," *Frontiers Neurosci.*, vol. 14, p. 119, Feb. 2020, doi: 10.3389/fnins.2020.00119.
[39] Y. Wu, L. Deng, G. Li, J. Zhu, Y. Xie, and L. Shi, "Direct training for spiking neural networks: Faster, larger, better," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 1311–1318.
[40] Y. Zhong, X. Hu, C. Luo, X. Wang, J. Zhao, and L. Zhang, "WHU-Hi: UAV-borne hyperspectral with high spatial resolution ($H^2$) benchmark datasets and classifier for precise crop identification based on deep convolutional neural network with CRF," *Remote Sens. Environ.*, vol. 250, Dec. 2020, Art. no. 112012. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0034425720303825
[41] A. L. Hodgkin and A. F. Huxley, "A quantitative description of membrane current and its application to conduction and excitation in nerve," *J. Physiol.*, vol. 117, no. 4, p. 500, 1952.
[42] L. F. Abbott, "Theoretical neuroscience rising," *Neuron*, vol. 60, no. 3, pp. 489–495, Nov. 2008.
[43] E. M. Izhikevich, "Simple model of spiking neurons," *IEEE Trans. Neural Netw.*, vol. 14, no. 6, pp. 1569–1572, Nov. 2003.
[44] Y. Liu, K. Cao, R. Wang, M. Tian, and Y. Xie, "Hyperspectral image classification of brain-inspired spiking neural network based on attention mechanism," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.
[45] Y. Venkatesha, Y. Kim, L. Tassiulas, and P. Panda, "Federated learning with spiking neural networks," *IEEE Trans. Signal Process.*, vol. 69, pp. 6183–6194, 2021.
[46] Y. Kim and P. Panda, "Visual explanations from spiking neural networks using inter-spike intervals," *Sci. Rep.*, vol. 11, no. 1, pp. 1–14, Dec. 2021.
[47] N. Rathi and K. Roy, "DIET-SNN: Direct input encoding with leakage and threshold optimization in deep spiking neural networks," 2020, *arXiv:2008.03658*.
[48] Y. Kim, H. Park, A. Moitra, A. Bhattacharjee, Y. Venkatesha, and P. Panda, "Rate coding or direct coding: Which one is better for accurate, robust, and energy-efficient spiking neural networks?" in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2022, pp. 71–75.
[49] M. E. Paoletti, J. M. Haut, R. Fernandez-Beltran, J. Plaza, A. J. Plaza, and F. Pla, "Deep pyramidal residual networks for spectral–spatial hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 740–754, Aug. 2018.
[50] Y. Bai, Q. Zhang, Z. Lu, and Y. Zhang, "SSDC-DenseNet: A cost-effective End-to-End spectral-spatial dual-channel dense network for hyperspectral image classification," *IEEE Access*, vol. 7, pp. 84876–84889, 2019.
[51] S. K. Roy, S. Manna, T. Song, and L. Bruzzone, "Attention-based adaptive spectral–spatial kernel ResNet for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 9, pp. 7831–7843, Sep. 2020.
[52] Z. Gong, P. Zhong, J. Qi, and P. Hu, "A CNN with noise inclined module and denoise framework for hyperspectral image classification," 2022, *arXiv:2205.12459*.
[53] T. Tieleman and G. E. Hinton, "Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude," *COURSERA, Neural Netw. Mach. Learn.*, vol. 4, no. 2, pp. 26–31, 2012.
[54] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
[55] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural network," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 1–9.
[56] D. Lin, S. Talathi, and S. Annapureddy, "Fixed point quantization of deep convolutional networks," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 2849–2858.



**Yang Liu** (Member, IEEE) received the B.S. degree from the Changchun University of Science and Technology, Changchun, China, in 1996, and the M.S. and Ph.D. degrees from Henan University, Kaifeng, China, in 2009 and 2016, respectively.

He is currently a Professor and a Ph.D. Supervisor with the College of Computer Science and Information Engineering, Henan University, where he is the Director of the Henan Province Engineering Research Center of Spatial Information Processing and a Principle Investigator of the Brain-Inspired Intelligence Science and Technology Innovative Team. He is mainly conducting research on the key algorithms of brain-inspired computing, such as cross-modal target recognition, cross-media semantic retrieval, multisource, and multimedia information processing. He has also developed complex systems of high-performance computing, such as spatiotemporal information processing platforms and remote sensing intelligent information extraction development kit. His research interests include the scientific theory of brain and mind-inspired computing (BMC), common technology of multimedia information processing, and engineering application of spatiotemporal big data. Specifically, he focuses on exploring the models of mind-like computing, such as multimedia neural cognitive computing and cross-modal cognitive neural computing.

Dr. Liu is a member of China Computer Federation (CCF) and Chinese Association for Artificial Intelligence (CAAI).



**Kejing Cao** received the bachelor's degree in computer science and technology from Henan University, Kaifeng, China, and Victoria University, Footscray, VIC, Australia, in 2020. She is currently pursuing the master's degree in software engineering with Henan University.

Her research interests include spiking neural networks, hyperspectral image classification, and brain-inspired computing.



**Rui Li** received the B.E. degree in network engineering from Henan Normal University, Xinxiang, China, in 2020. She is currently pursuing the M.E. degree in computer science and technology with Henan University, Kaifeng, Henan, China.

Her research interests include spiking neural networks and cross-modal recognition.



**Hongxia Zhang** received the B.E. degree in software engineering from Zhoukou Normal University, Zhoukou, China, in 2021. She is currently pursuing the M.A. degree in software engineering with Henan University, Kaifeng, China.

Her research interests cover cross-media recognition and spiking neural networks.



**Liming Zhou** received the Ph.D. degree from the State Key Laboratory of Networking and Switch Technology, Beijing University of Posts and Telecommunications, Beijing, China, in 2015.

He has been an Associate Professor with the School of Computer and Information Engineering, Henan University, Kaifeng, China, since 2015. His research interests include artificial intelligence, target detection, and information security.